# Lesson 14 Teacher's Key: Raising the Stakes

GDevelop - Instructor Reference Only

VIDEO GAME CREATION

## What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as GDevelop expressions inside the Setup and advance-wave events built since Lesson 10/11.

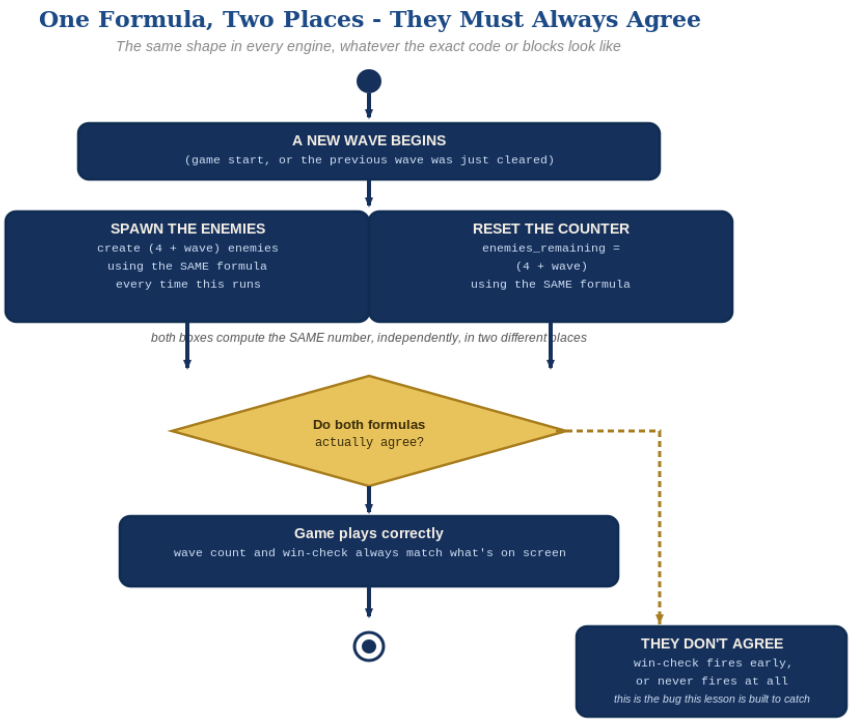
## Level 1 - Full Worked Solution

- (4 + Wave) as a repeat count and as EnemiesRemaining's reset value must be identical in BOTH the Setup event group and the advance-wave event - two places, one formula, each independently editable, so both need checking.
- No new variables are needed for Level 1 - this is purely an expression change inside events that already exist.
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.

*Highlighted lines are new this week; everything else is code you already have.*

```
Setup event group (revised):
Action: Repeat (4 + Wave) times -> Create object Enemy
Action: EnemiesRemaining = 4 + Wave

Advance-wave event (revised, from Lesson 11):
Action: Wave = Wave + 1
Action: EnemiesRemaining = 4 + Wave
Action: Repeat (4 + Wave) times -> Create object Enemy
```




*Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.*

VIDEO GAME CREATION

*Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.*

## Why This Structure

- GDevelop's expression fields accept the SAME formula ((4 + Wave)) typed directly wherever a number is expected - no separate 'store it in a variable first' step is required the way GameMaker's or Unity's code needed, though it's still good practice to double check both events agree.
- For Level 2, adding a Speed object variable to Enemy (rather than a scene-wide variable) keeps each enemy's speed as its own property, consistent with how Score/Lives/Wave are scene-wide but per-enemy state (like position) has always lived on the object itself.

## Level 2 / Level 3 Talking Points

- Ask students to find both places (4 + Wave) needs to appear (Setup group, advance-wave event) and confirm they're both actually updated - a good comprehension check for events versus a single function.
- For students comparing engines: point out that Scratch's identical (4 + wave) formula, expressed as blocks instead of GDevelop's expression fields, is conceptually the exact same idea.

**Level 2 - complete code:**

*Highlighted lines are the Level 2 additions on top of the Level 1 code above.*

```
Setup event group (Level 2 addition, on top of the Level 1 code above):
Action: Repeat (4 + Wave) times -> Create object Enemy
Action: EnemiesRemaining = 4 + Wave
Action: Change Speed of Enemy to (30 + (Wave - 1) * 10)

Advance-wave event (Level 2 addition, same change):
Action: Wave = Wave + 1
Action: EnemiesRemaining = 4 + Wave
Action: Repeat (4 + Wave) times -> Create object Enemy
Action: Change Speed of Enemy to (30 + (Wave - 1) * 10)

Enemy's existing downward-movement action (revised to read the variable):
Action: Change Enemy's Y position by Enemy.Variable(Speed) (replaces the
old fixed number)
```

**Level 3 - no single combined code example:**

*This week's Level 3 is a "choose ONE" menu of five independent stretch features (destructible shields, a power-up, a bonus enemy, formation-marching movement, or a title/game-over screen) rather than the usual set of small, combinable stretch goals - so a single "all goals combined" code block doesn't fit Rule 14's usual model, and isn't provided here. Full worked solutions for all five options, across the active engines, are planned for the Lesson 15 wrap-up "Complete Codebase" send-home materials instead. Use the Talking Points above and the Handout's Level 3 menu to guide students toward whichever option they choose.*

## Common Student Mistakes

- Updating the Setup group's formula but forgetting the advance-wave event's matching formula (or vice versa) - these are two separate events since Lesson 11, easy to update only one.
- Adding Speed as a scene variable instead of an object variable on Enemy, which would give every enemy the SAME speed rather than each one carrying its own.
- Forgetting to change the Enemy's movement action to read Enemy.Variable(Speed), leaving it at a hardcoded number.